Final Project Report

Elevator Operating System Scheduler

1. Scheduling Strategy

1.1 Algorithm Overview

The scheduler is a greedy, score-based heuristic. When someone shows up in the input queue, the dispatch thread looks at every elevator that can actually reach their floors, scores each one, and picks the lowest. Lower scores mean better matches.

1.2 Score Computation

The score is a sum of five terms, none of them particularly clever:

- **gap × 2** — distance in floors between the elevator and the pickup. We doubled it so distance outweighs everything else. Every floor of travel costs real timesteps, so we didn't want a small load advantage to send someone past a closer elevator.
- **busy** = `riders × 3 + pending × 4`. *Riders* comes from the latest `/ElevatorStatus` poll; *pending* is what we've already assigned but who hasn't boarded yet. We weighted pending a little higher than riders on purpose — it kept us from dogpiling new assignments onto an elevator that was already committed but hadn't started moving people yet.
- **wrong_dir** is just a flat 6-point hit if the elevator is heading the wrong way.
- **span** = `(high − low) ÷ 10`. A small penalty for wide-range elevators. The point isn't to discourage them generally — it's to keep them free for the trips only they can serve.
- **here**: 3 points off if the elevator is already on the pickup floor. Boarding can start immediately, no travel cost.

We didn't tune the weights with anything formal. We started with educated guesses, watched the runs, and nudged things until the load distribution looked sane.

1.3 Decision Flow Over Time

Decisions happen continuously as new requests come in. The flow looks something like this:

1. The input thread polls `/NextInput` in a tight loop. When it gets a person back, it parses the response and pushes a `Person` struct onto a thread-safe queue.
2. The dispatch thread pops the queue and runs the scoring across all in-range elevators. The winner gets committed via `PUT /AddPersonToElevator`, and we bump the pending counter for that elevator.
3. Independently, the status thread polls `/Simulation/check` and `/ElevatorStatus/<id>` for every elevator on a 250 ms cadence. When a poll shows more riders than the previous one, we treat the difference as boardings and decrement pending by that much.

1.4 Hard Capacity Check

The hard capacity gate inside the scoring loop was a late addition, and a critical one. Before scoring an elevator, we check whether its `riders + pending` already meets or exceeds its capacity. If so, we skip it entirely.

We added it after digging into the simulator's source on GitHub. Turned out that when an elevator gets to a person's pickup floor and is already at capacity, the simulator yanks them off the elevator's pickup list and drops them back into the input queue. Without the gate, our scheduler would re-pull that same person from `/NextInput` and assign them right back to the same overloaded elevator — a livelock that ate hundreds of timesteps.

When every range-compatible elevator is at capacity at once, we fall back to the least-loaded option rather than dropping the person. Keeps things moving forward instead of retrying forever.

1.5 Why a Greedy Approach

We went with a simple greedy approach because it fit the situation. The scoring loop is O(n) per assignment with n = 8 elevators, so the cost is rounding error next to the HTTP calls. The API only hands us one person at a time anyway — there is nothing to look ahead at. And the simulator's elevator already runs a Scan-style sweep internally, which means stop ordering inside an elevator is solved; our job is just deciding which elevator gets which person.

A globally optimal scheduler would mean treating this as an assignment problem and solving it across all in-flight requests and would be more complex.

2. Design Decisions

2.1 Overall System Design

It's a single C++ process. Talks to the Python Elevator Operating System over plain-text HTTP on localhost, keeps all state in memory (nothing on disk), and centers everything around a scheduler class that owns the building config, the elevator state, and the three workers handling input, dispatch, and polling.

The HTTP client is hand-rolled on top of POSIX sockets. The API only does get and put with plain-text responses, so we got away with writing more code.

2.2 Thread Structure and Responsibilities

We use three worker threads beyond main, each with one job:

- The **input thread** (`input_worker`) grabs new ride requests. It polls `/NextInput` in a tight loop. When it gets a person back, it parses the pipe-delimited response and pushes a `Person` struct onto the shared queue. If the response is `"NONE"` (nothing available right now), it sleeps 50 ms before polling again.
- The **dispatch thread** (`dispatch_worker`) assigns people to elevators. It pops the queue (with a 250 ms timeout so it can re-check the shutdown flags), runs the scoring, and fires off `PUT /AddPersonToElevator/<personID>/<elevatorID>`. If the request fails, it retries up to eight times before dropping the person and logging it.
- The **status thread** (`status_worker`) keeps our local view of elevator state in sync with the simulator. It polls `/Simulation/check` to know when the sim ends, and while the sim's running, it polls `/ElevatorStatus/<id>` for every elevator and updates the in-memory table.

2.3 Data Flow Between Threads

The threads share three things. New ride requests flow from the input thread to the dispatch thread through RideQueu, a thread-safe FIFO. Live elevator state flows from the status thread into a shared map (`status`), which the dispatch thread reads while scoring. Termination signals flow between all three threads through a small set of atomic boolean flags.

The dispatch thread occasionally pushes back into RideQueu itself, when an assignment fails and needs a retry — so it's both a consumer and an occasional producer.

2.4 Synchronization

There are three synchronization tools in play, each picked for a different problem.

- A single `std::mutex` called `state_lock` covers the two shared maps (`status` and `pending`). Anything that touches either map grabs it through a `std::lock_guard`. Critical sections are short, so we never saw any real contention.
- `RideQueue` has its own mutex and a `std::condition_variable`. The condition variable lets the dispatch thread sleep cheaply when the queue is empty — without it we'd be busy-looping or sleeping on a fixed timer, which would either waste CPU or add latency. We signal it when something gets pushed or when the queue closes during shutdown.
- Three `std::atomic<bool>` flags (`running`, `sim_done`, `input_done`) handle cross-thread termination. Each flag has one writer and a few readers, which is the use case atomics were made for. A mutex would have worked too, but it would have been more code with no real benefit.

The static building config (the `elevators` vector) gets populated once during construction and never changes after, so we read it from any thread without locking. There is an inline comment in the function that does the lockless read, so a future maintainer doesn't think we forgot a lock.

3. Testing and Validation

3.1 Correctness Verification

We compiled the scheduler locally with g++ -std=c++17 -Wall -Wextra -pthread -O2 (no warnings) and rebuilt it on the HPCC cluster under gcc/14.2.0 to match the grading environment. We verified correctness by running the official highrise_busy_grader.sh against the live simulator.

The final version scored 100% on Code Efficiency and 100% on Correctness for highrise_busy. All 300 people delivered in 1,271 timesteps — well under the 1,401 threshold for max efficiency. Zero people left in the input queue, zero waiting for an elevator, zero stranded on board at the end.

We also ran it against the other three scenarios in the project's `submission_scripts` directory. All three finished cleanly:

- **highrise_slow** — 100 of 100 trips in 551 timesteps. Nothing left over.
- **hotel_busy** — 50 of 50 in 259 timesteps. Nothing left over.
- **hotel_slow** — 10 of 10 in 111 timesteps. Nothing left over.

Across all four scenarios — 460 trips total — every person made it to their destination before the sim ended. Nothing deadlocked, nothing hung, and we never saw a late-stage assignment failure.

3.2 Determinism Across Multiple Executions

We enforce determinism inside pick_elevator with an alphabetical tiebreaker on bay name when two elevators get identical scores. The rubric flags non-deterministic behavior as incorrect, so we treat this as a correctness requirement, not a nice-to-have one. Running the same workload multiple times produced identical assignment sequences.

3.3 Major Bugs and How We Fixed Them

We passed the grader on the third try. The first two attempts both scored 0%, and it was intereseting the way we got to the 100%.

Iteration 1 — 0% / 0%, 109 of 300 completed. Our first full run hit the simulator's 1542-timestep cap, with 119 people still waiting for an elevator and 72 stuck on board. The trace logs showed the same six names — Frances, Joshua, Sarah, Suzanne, Larry, Scott — getting reassigned in a tight loop. Digging in, we found the cause: when the simulator's elevator arrived at someone's pickup floor and was already at capacity, it would pull the person off the pickup list and drop them back into/NextInput. Our scheduler would then re-pull them and re-assign them, bumping pending each time, with no boarding event coming in to bring it back down. Pending grew without bound, and the per-elevator load

came out badly : the two narrow-range elevators (HighriseG, HighriseH) each carried 41 trips, while one of the wides (HighriseD) carried zero.

Iteration 2 — 0% / 0%, 52 of 300 completed. We misread the first failure and pulled out the span_cost term, thinking it was unfairly penalizing the wide-range elevators. The score got worse — from 109 completed down to 52. Turns out span_cost wasn't a bug, it was a crude load-balancer: by giving narrow elevators a small edge on short trips, it kept the wides free for the upper-floor traffic only they could handle. Removing it spread short trips evenly across all eight elevators, which starved the wides of throughput for trips that *required* them. We also capped pending at capacity in this iteration, but without the hard skip gate to actually use the cap, it didn't help.

Iteration 3 — 100% / 100%, 300 of 300 completed. We reverted the scoring function (put span, here -=3, original busy weights) and added three guardrails around it instead: a hard capacity skip in pick_elevator that won't even score an elevator whose Riders + Pending already meets capacity; a fallback to the least-loaded in-range elevator when every candidate is full; and a clamp on pending at capacity so re-queue cycles can't inflate it past the physical limit. Same workload, all 300 trips completed, with the per-elevator load flattening out to between 34 and 43 trips each.

The takeaway was that the scoring trial we did was somewhat right from day one. What was missing was the consideration for physical capacity. The fix wasn't to rewrite the algorithm but was to add the gates that stop it from making impossible commitments.

4. Performance Considerations

4.1 Minimizing Total Simulation Timesteps

The rubric scores efficiency on one number: total simulation timesteps to deliver everyone. A few of our choices were aimed directly at that number.

The biggest one is the hard capacity gate from earlier. By refusing to assign people to elevators that physically can't take them, we sidestep the re-queue hole that ate timesteps in our early runs. Once the gate was in, the elevator load distribution flattened out across all eight elevators in the highrise scenario — each one carried between 34 and 43 cumulative riders, versus the pre-fix split where two elevators carried 41 each while another carried zero.

The polling measure are tuned for throughput too. The input thread sleeps just 50 ms between consecutive NextUnput polls when the response is “none” , short enough that the gap between an arrival becoming available and us reacting to it doesn't delay the elevator's next move. The status thread polls every 250 ms, fast enough to catch

boardings within one or two elevator-floor transitions but slow enough to not hammer the simulator's HTTP server.

4.2 Tradeoffs Between Simplicity and Efficiency

There are a couple of places where we picked the simpler thing over the faster thing.

The HTTP client is hand-rolled instead of built on cURL and it doesn't pool connections, every API call opens a fresh TCP socket, sends one request, reads one response, and closes. That adds a small overhead per call, but it kept the connection lifecycle simple and saved us from carrying an external library around.

The scheduling itself is greedy and per-person rather than a global assignment over the queue. A global solver would in theory save more timesteps by considering combinations, but it would mean buffering requests, re-running the optimization on every change, and adding a lot more code. The greedy version was enough to hit best efficiency on the available scenario, so we stopped there.

And also one mutex covers both the elevator status map and the pending counter map, instead of finer-grained locks. The two maps tend to get touched together anyway, and the critical sections are short enough that there is no contention worth optimizing around.

5. Limitations and Improvements

5.1 Known Limitations

The greedy approach has the obvious downside: it can pick locally well but globally badly. If two range-compatible elevators score similarly, we just pick one — we don't think about whether the other might be a better fit for someone arriving thirty seconds from now. A clairvoyant scheduler would sometimes choose differently. On these workloads, the difference wasn't enough for us to go after.

The pending counter is also an approximation. We bump it up when an assignment goes through and bump it down when an elevatorStatus poll shows the rider count climb — but the link between the assignment and the actual boarding is inferred, not directly observed.

5.2 Future Improvements

If we kept working on this, there are a few things we'd clean up next. The scoring weights were tuned by inspection rather than any real search — a small offline loop that

ran the scheduler against the test scenarios with different weight configurations would probably squeeze a few more timesteps out. The HTTP client could grow persistent connections with connection: keep-alive, which would kill the per-request socket setup cost. And we could buffer arrivals briefly and consider them in small batches, so each assignment factors in near-term load instead of reacting to one person in isolation.

Appendix: Individual Contributions

Aaron Medina

Assigned Role(s)

I had two roles on the project: Project Manager and Documentation & Analysis Lead. As PM, I coordinated hand-offs between the two of us, and made sure each deliverable — the first weekly check-in, and the final report, the submission itself — was ready ahead of its deadline. We didn't really use much of the GitHub repo, as we mostly worked on the project together in person using one computer. As the documentation lead, I mostly wrote the weekly check-ins and the unified final report, translating the technical work into something readable and keeping the docs aligned with what was actually in the repo.

Contributions

On the project management side, I coordinated the hand-off between Gerardo and myself, lined up the code, the report, and the check-in document before the deadline, and wrote each of the weekly check-ins.

On the documentation side, I wrote Sections 1 through 5 of this report — the scheduling strategy, the three-thread design, the scoring heuristic and why its weights are what they are, the testing and validation results, performance, and known limitations. I pulled the testing notes and per-elevator load data from Gerardo's QA work into the results section, and I wrote up the iteration history (the 0% / 0% / 100% sequence) so it actually explains the technical lesson instead of just listing outcomes. I also did the final check-in pass, making sure each claim in it matched the work and the project board before submission.

Technical Understanding

The scheduler is a single C++ process that talks to the Python simulator over a plain-text HTTP API on localhost, organized around three worker threads. The input thread polls NextInput and pushes new requests onto a thread-safe inbox queue. The *dispatch thread* pops from that queue and runs the scoring heuristic to decide which elevator gets each rider. The *status thread* polls every elevator on a 250 ms cadence to keep our local view of rider counts and positions in sync with the simulator. The three threads share state through a single mutex over the elevator status and pending-counter maps, plus a condition variable on the inbox so the dispatch thread can sleep cheaply when there's no work.

The scoring heuristic in pick_elevator is the brain of the dispatch thread. It sums five terms — distance to the rider's pickup (doubled, because each floor of travel costs real timesteps), current load from riders plus pending assignments, a wrong-direction penalty, a small range-span penalty that nudges short trips toward narrower elevators, and a same-floor pickup bonus, and picks the lowest-scoring eligible elevator. The weights were tuned against the test scenarios. The most important property of the function is that the `span` term works as a crude load balancer, keeping the wide-range elevators free for the trips only they can handle. Wrapped around the scoring is a hard capacity gate that skips any elevator already at riders + pending >= capacity, with a fallback to the least-loaded option when everyone's full.

Challenges and Resolutions

The biggest documentation challenge was keeping the report in sync with code that was actively changing. During the three-iteration sequence that took us from 0% to 100% on the grader, both the scoring function and the capacity-handling logic shifted between drafts. I ended up rewriting Section 1.4

and the Section 3.3 iteration narrative twice as Gerardo's fixes landed. The fix on my end was to stop writing about specific numbers until the iteration that actually scored 100%, then go back and fill in the per-iteration completion counts and load distributions from the captured logs.

The second challenge was coordinating a two-person team where one person owned all the implementation. It would've been easy for the documentation to drift from the code if I hadn't actively pulled status from Gerardo on each meaningful commit, so I made a habit of walking through every change with him and updating the project report and etc… By the final week the board, and the report were fully consistent, which made the last check-in straightforward to write and verify.

“I certify that the submitted code was written by myself or collab with my group members as described in this document. I did not use AI-generated code or unauthorized external sources in the development of this project. I understand that the use of such materials constitutes academic dishonesty and will result in failure of the course.”

Gerardo Jimenez

Assigned Role(s)

I had two roles on the project, I was lead developer and QA/Verification Lead. As Lead developer, I was on the hook for schedule_os end-to-end, the threading model, the HTTP client, the scoring heuristic, the synchronization layer. As QA Lead, I ran the scheduler against the official grader, validated its behavior across multiple test scenarios, confirmed deterministic output across re-runs, and diagnosed every failure we hit along the way.

Contributions

On the dev side, I designed and wrote the three-thread architecture status, input and dispatch worker inside the Scheduler class, including the producer-consumer queue (RideQueue) that connects the input thread to the dispatch thread through a std::mutex and std::condition_variable. I wrote the hand-rolled HTTP/1.1 client on raw POSIX sockets — connect_loopback, send_all, recv_all, parse_resp, and http — covering both the request side and response parsing without an external library. I built the scoring heuristic in score, including all five terms gap, busy, wrong_dir, span, here and tuned the weights against the test scenarios. I also wrote the building file parser in load_building, the helper utilities trim and split for parsing the simulator's pipe-delimited responses, and the SIGPIPE-handling at startup that keeps the kernel from killing the process when a sim socket closes mid-write.

On the QA side, I wrote the Makefile, ran every iteration of the grader against the highriser_busy_grader.sh, captured the timestep and people-remaining numbers from each run, and tracked the iteration history that eventually got us to 100% / 100%. I also verified determinism by running the same workload several times and confirming the assignment sequences matched, and I dug through the report logs to confirm the capacity gate was producing the balanced load distribution we wanted.

Technical Understanding

The pieces I worked on are tightly interconnected. The three worker threads each have one responsibility, but they share state through the state_lock mutex the status thread writes to the elevator state map after each elevatorStatus poll while the dispatch thread reads from the same map when

scoring candidates. The producer-consumer relationship between the input thread and the dispatch thread runs through rideque, which uses a condition variable so the consumer can sleep cheaply when there's nothing to dispatch. Termination across all three threads is coordinated through three atomic boolean flags running, sim, input; each flag has one writer and multiple readers, which is what std-atomic is built for.

The scoring function is the brain of the dispatch thread. Each term targets a specific consideration — distance to pickup, current load, direction match, range tradeoff, same-floor pickup. The hard capacity check that wraps the scoring loop is what made the heuristic actually work in practice: the simulator re-queues people who try to board a full elevator, and without the gate, the scheduler just kept re-assigning the same people in a livelock. The fallback to least-loaded when everyone's at capacity makes sure the dispatch thread doesn't come up empty for someone whose only problem is timing.

Challenges and Resolutions

The biggest challenge was diagnosing why the first two grader runs scored 0%. The first run completed only 109 of 300 trips, and the trace logs showed the same six riders getting reassigned in a tight loop. My first response was that span_cost was unfairly penalizing the wide-range elevators — turned out to be wrong, pulling it dropped the completion count to 52, because span_cost was actually working as a crude load-balancer that kept the wides free for trips only they could handle.

The real problem only got clear after I read the Python simulator's source on GitHub and found the line that re-queues people back into nextinput when they try to board a full elevator. Once I understood the re-queue cycle, the fix was straightforward: a hard capacity gate in pick_elevator that won't even score a full elevator, a fallback to the least-loaded option when every candidate is full, and a clamp on the pending counter so re-queue cycles can't push it past the elevator's physical capacity. Iteration three scored 100% / 100%, with the per-elevator load distribution evening out across all eight elevators.

“I certify that the submitted code was written by myself or collab with my group members as described in this document. I did not use AI-generated code or unauthorized external sources in the development of this project. I understand that the use of such materials constitutes academic dishonesty and will result in failure of the course.”